SLIIT
Discover Your Future

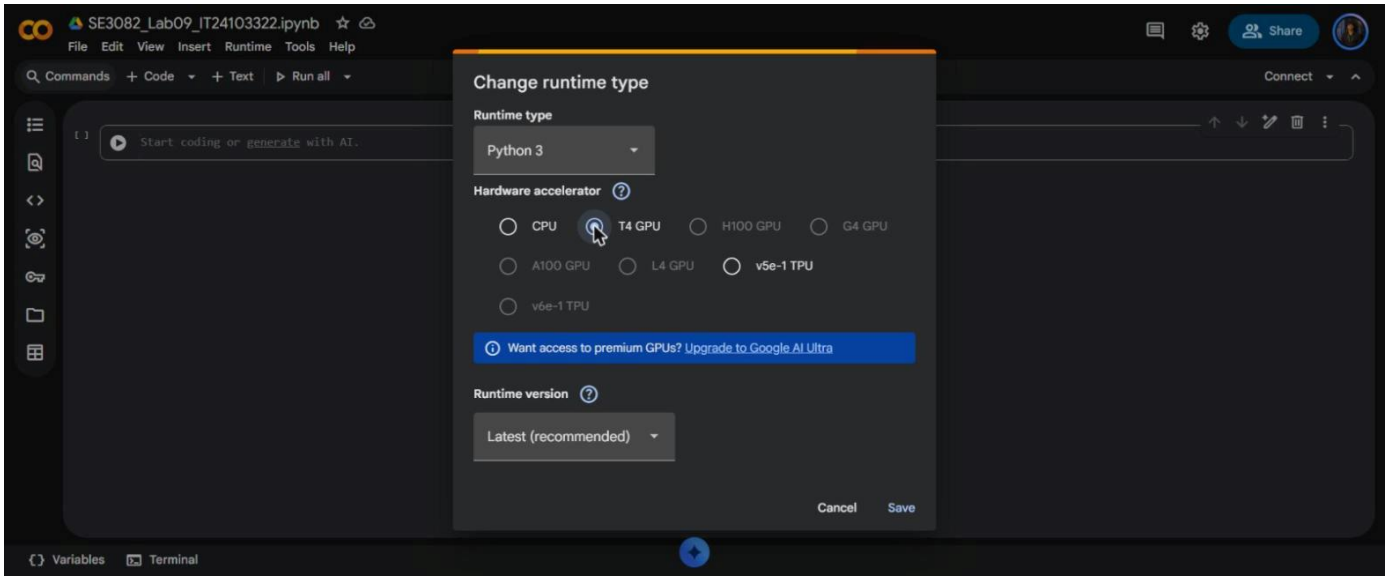
BSc (Hons) in Computer Science
Year 3

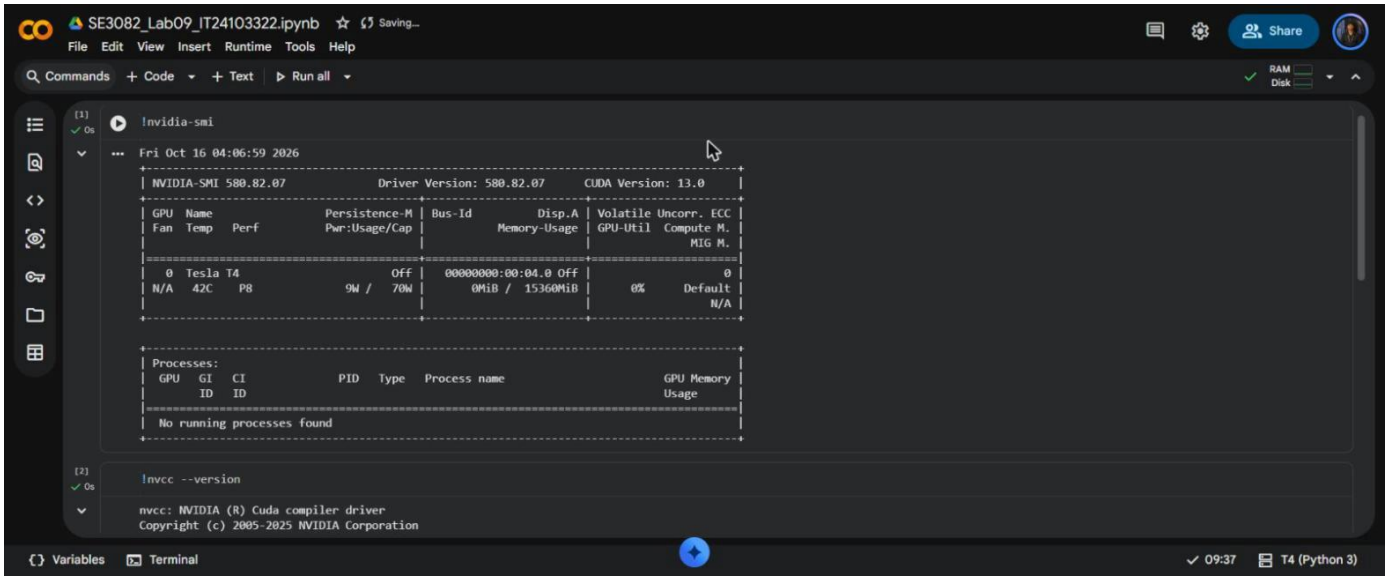
Lab Sheets

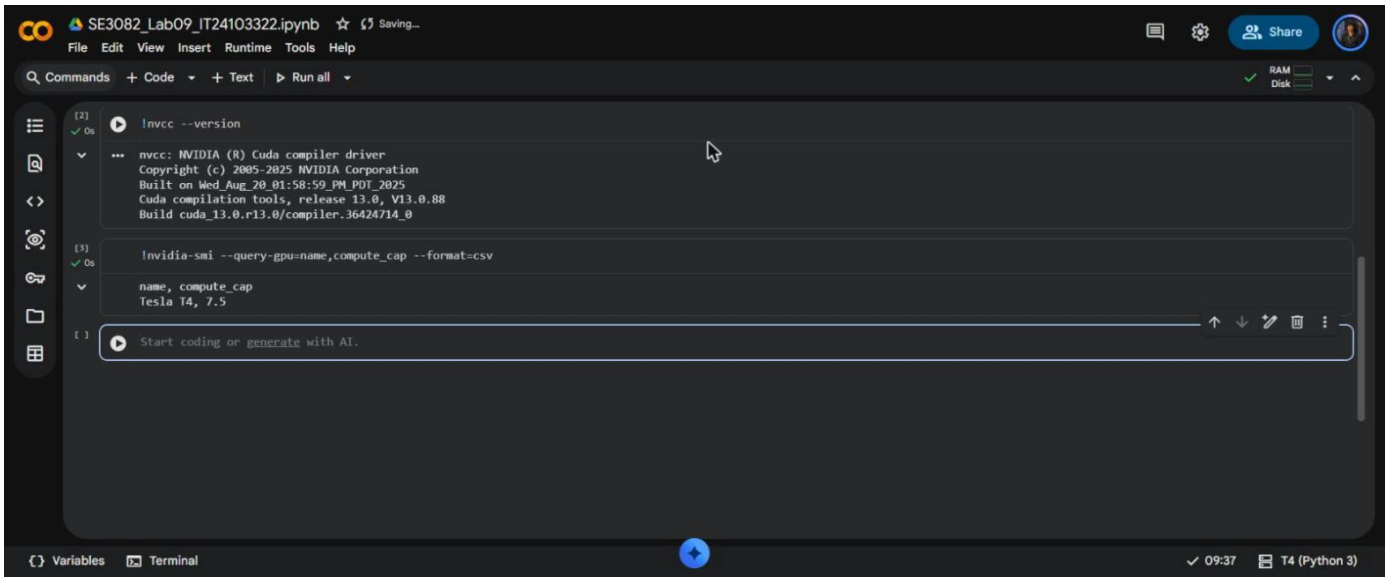
SE3082 – Parallel Computing **Semster 1, 2026**

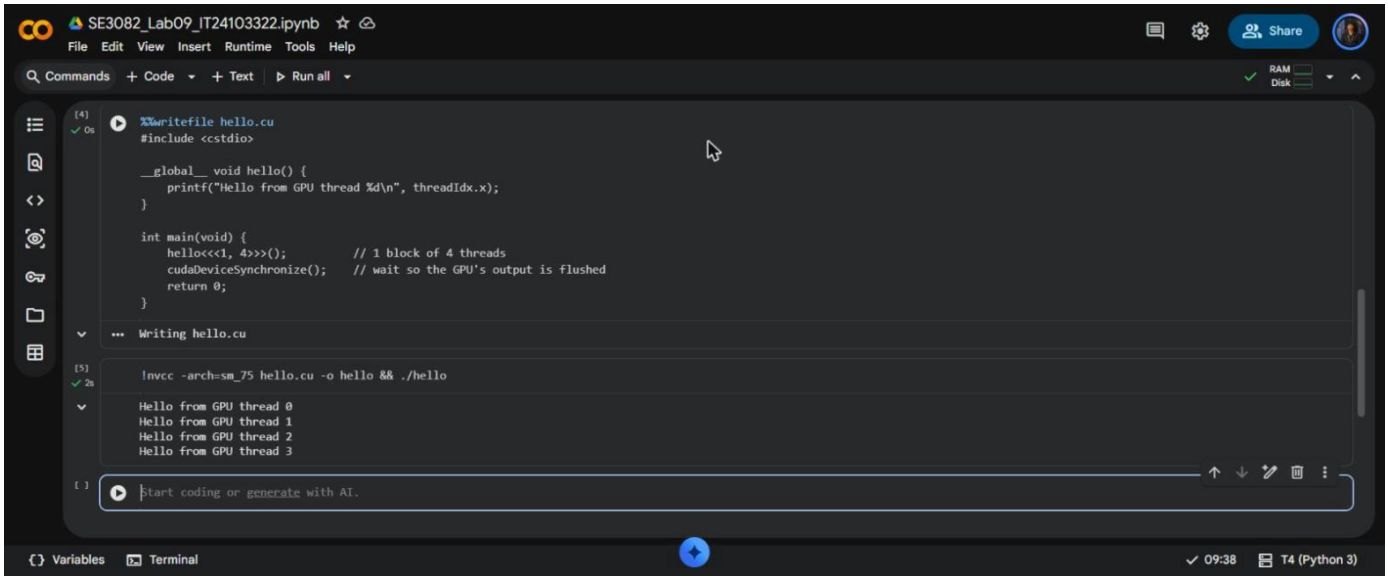
IT24103322

Exercise 1







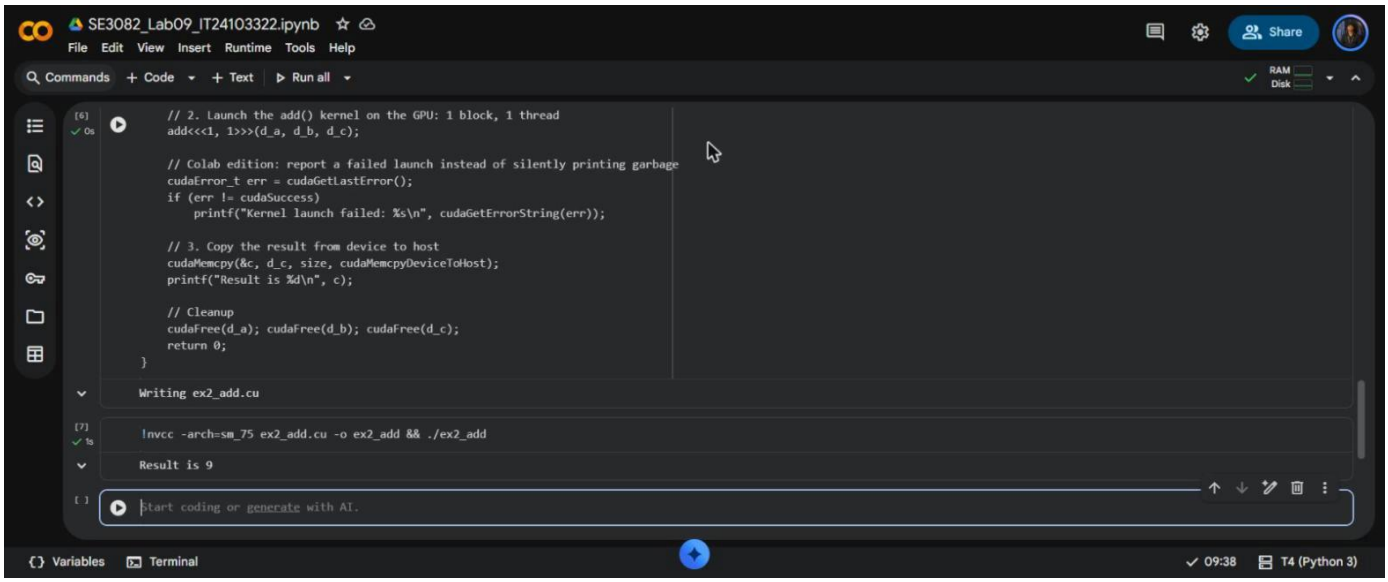


A new notebook was created in Google Colab and renamed to SE3082_Lab09_IT24103322. Under Runtime → Change runtime type the hardware accelerator was set to T4 GPU and saved.

!nvidia-smi shows the GPU: Tesla T4 with 15360 MiB of memory (driver 580.82.07, CUDA Version 13.0). !nvcc --version shows Cuda compilation tools, release 13.0 (V13.0.88), and the compute_cap query returns Tesla T4, 7.5, so the compiler flag is -arch=sm_75, as used in every compile cell.

hello.cu was written with %%writefile, compiled with nvcc and run. hello<<<1, 4>>>() launches one block of 4 threads, so four lines are printed, from GPU thread 0 to thread 3. cudaDeviceSynchronize() makes the CPU wait until the kernel has finished, so the GPU's printf output is flushed before the program ends. The order of the lines is not fixed, because the four threads run at the same time.

Exercise 2



The program adds two integers on the GPU with one block of one thread. a = 2 and b = 7 are copied from the host to the device with cudaMemcpy(..., cudaMemcpyHostToDevice), the add kernel is launched with add<<<1, 1>>>, and the result is copied back with cudaMemcpy(..., cudaMemcpyDeviceToHost). Output: Result is 9. No “Kernel launch failed” line was printed, so cudaGetLastError() returned cudaSuccess.

1. **Why do we need separate variables d_a, d_b, d_c on the device? Why can the kernel not use a, b, c directly?**

   The CPU and the GPU have separate memories. a, b and c live in host (CPU) memory, and the kernel runs on the GPU, which can only read and write device memory. d_a, d_b and d_c point to memory allocated on the GPU with cudaMalloc, so the inputs have to be copied into them before the kernel runs and the result has to be copied back afterwards.

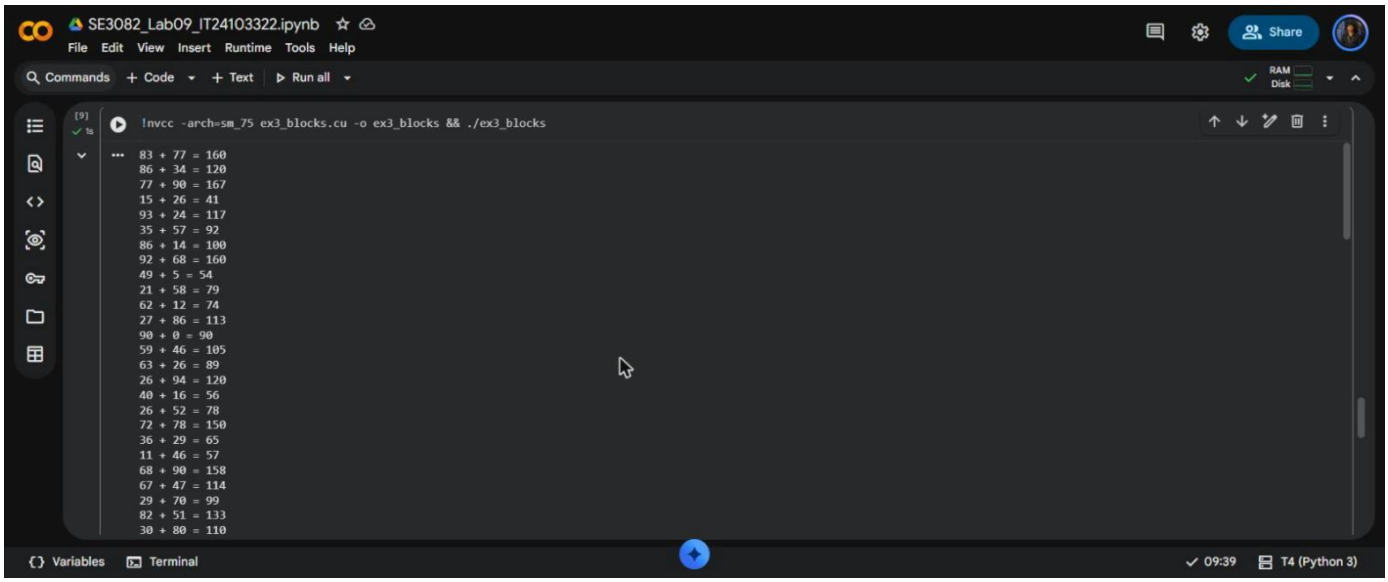
2. **In cudaMemcpy(d_a, &a, ...) we pass &a, but in the kernel launch we pass d_a. Why?**

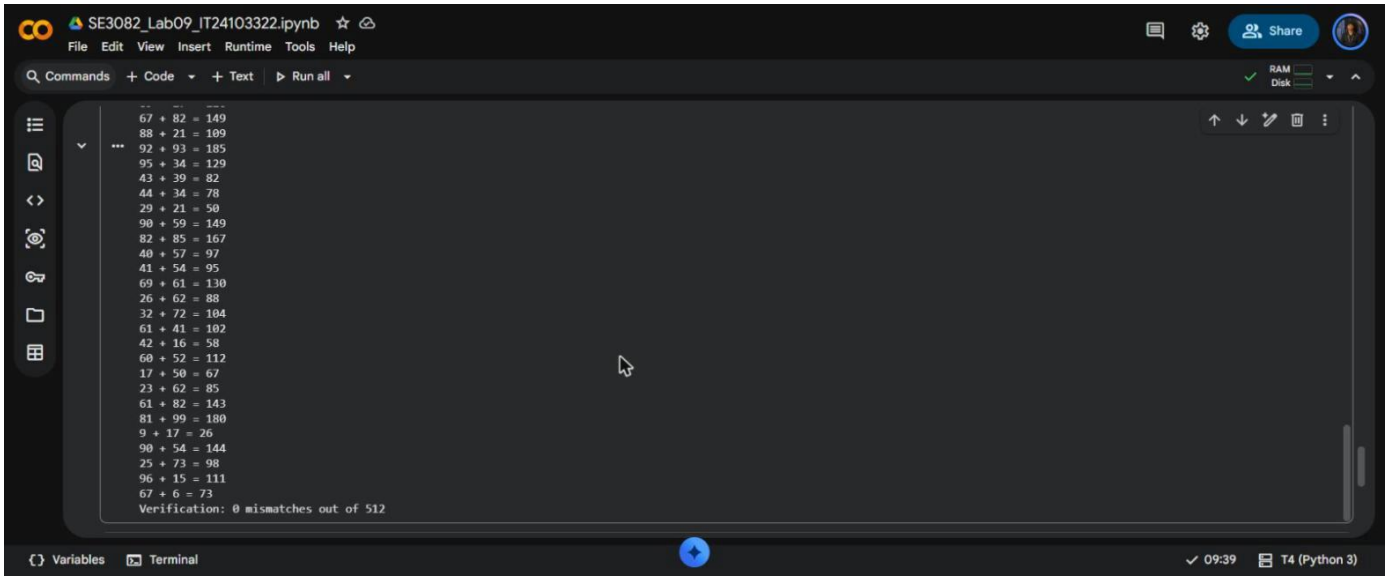
   cudaMemcpy needs the address of the source and the destination. a is a normal int, so &a is its address in host memory, and d_a is already a pointer to device memory. The kernel takes pointers to device memory (int *a), so it is given d_a, the GPU address, and never a host address.

3. **How many threads run the add kernel? What would you change to run it with 10 threads, and would the answer change?**

   add<<<1, 1>>> runs exactly one thread. To run it with 10 threads the launch becomes add<<<1, 10>>>. The answer stays 9, because every thread does the same *c = *a + *b with the same inputs and writes the same value to the same address; the extra threads only repeat the same work.

Exercise 3





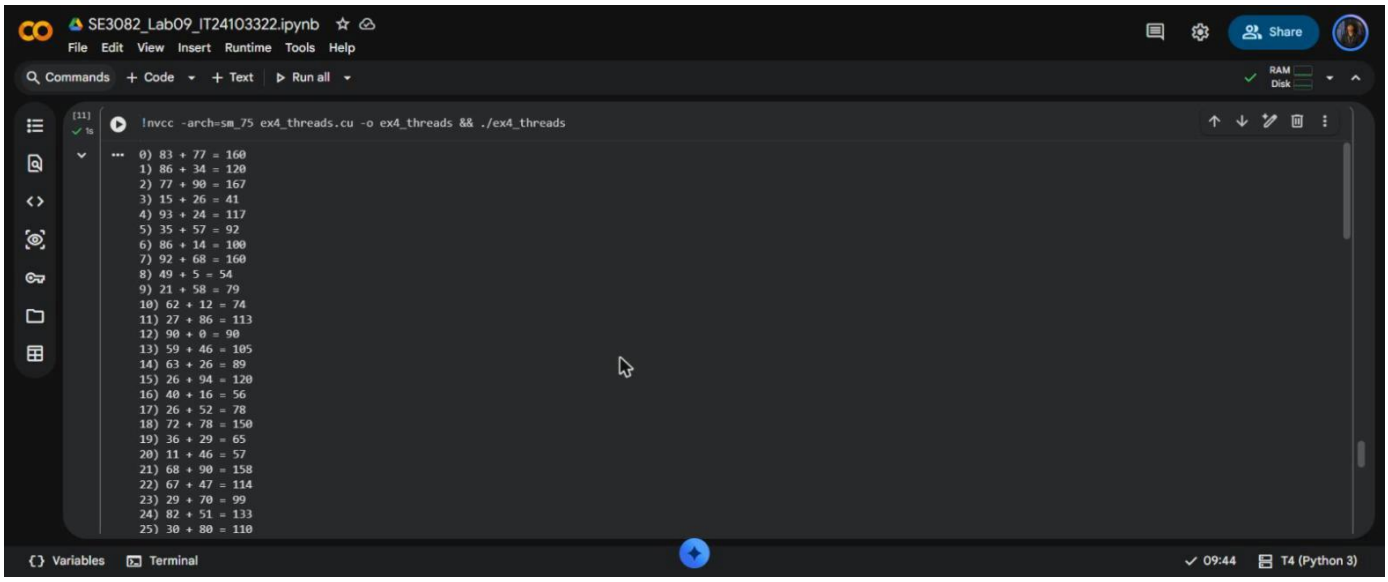
Two vectors of 512 integers (rand() % 100) are added with add<<<N, 1>>>, so the kernel runs in 512 blocks with one thread each and every block uses blockIdx.x to pick its element: c[blockIdx.x] = a[blockIdx.x] + b[blockIdx.x].

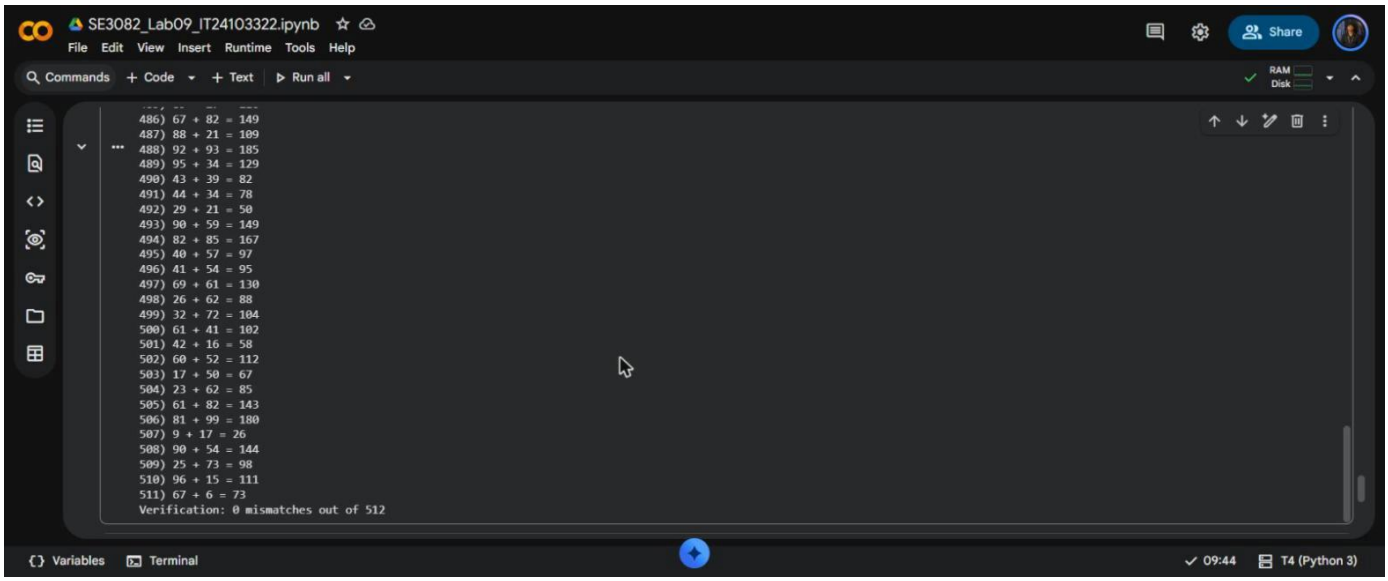
The first lines are 83 + 77 = 160, 86 + 34 = 120 and 77 + 90 = 167, as in the lab sheet, and the last line is Verification: 0 mismatches out of 512, so every GPU result matches the CPU.

1. **How many blocks and how many threads in total does add<<<N, 1>>> launch? What does a warp of 32 mean for a block that contains only one thread?**

   It launches 512 blocks and 512 threads in total (512 × 1). The GPU runs threads in warps of 32, and a warp never contains threads from two different blocks. A block with one thread still takes a whole warp, so 1 lane does the work and the other 31 lanes are idle: only 1/32 of the GPU's lanes are used, which is why one thread per block is very inefficient.

Exercise 4





The same addition with one block of 512 threads, addT<<<1, N>>>. Each thread uses threadIdx.x to choose its element. The first line is 0) 83 + 77 = 160, the last result is 511) 67 + 6 = 73 and the final line is Verification: 0 mismatches out of 512, as expected. The values are the same as Exercise 3 because the same rand() sequence is used.

1. **Change #define N 512 to #define N 2048, compile and run again. What is printed, which part of the program told you, and which limit of the GPU did you hit?**



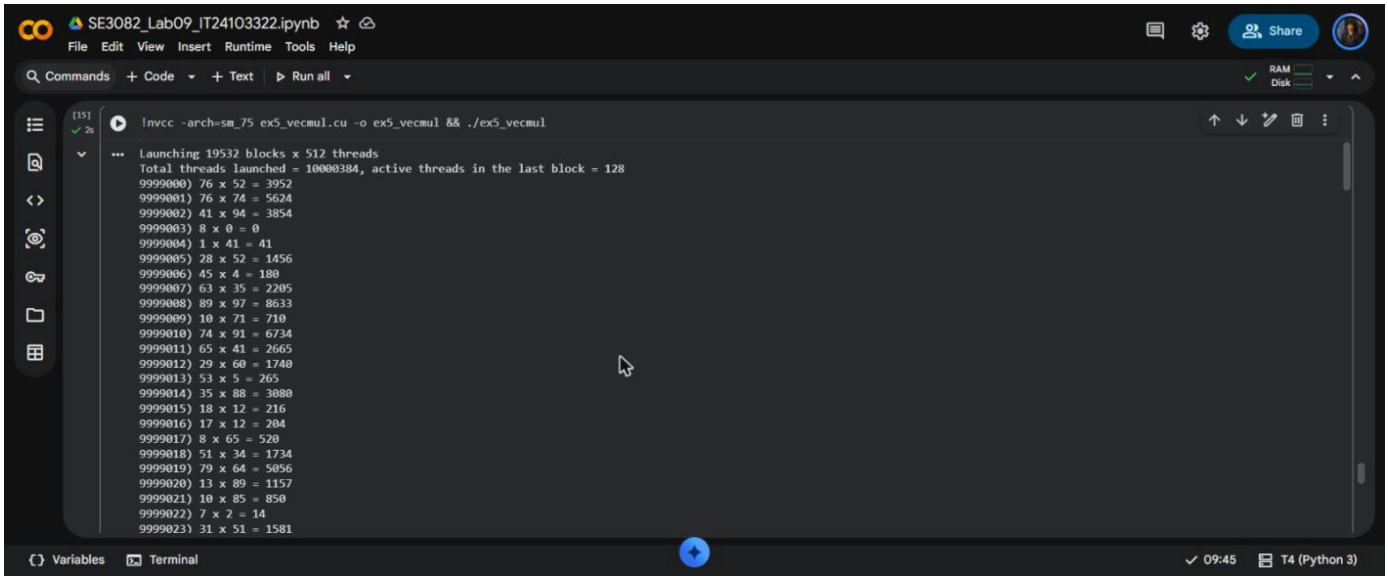
To keep ex4_threads.cu unchanged, the cell copies it to ex4_2048.cu with N changed to 2048 (sed), then compiles and runs it, printing the first and last lines of the output. The program printed Kernel launch failed: invalid argument. This line comes from the cudaGetLastError() check after the launch; without it the program would carry on silently. Because the kernel never ran, every c value is 0 (for example 0) 83 + 51 = 0) and the result is Verification: 2048 mismatches out of 2048.

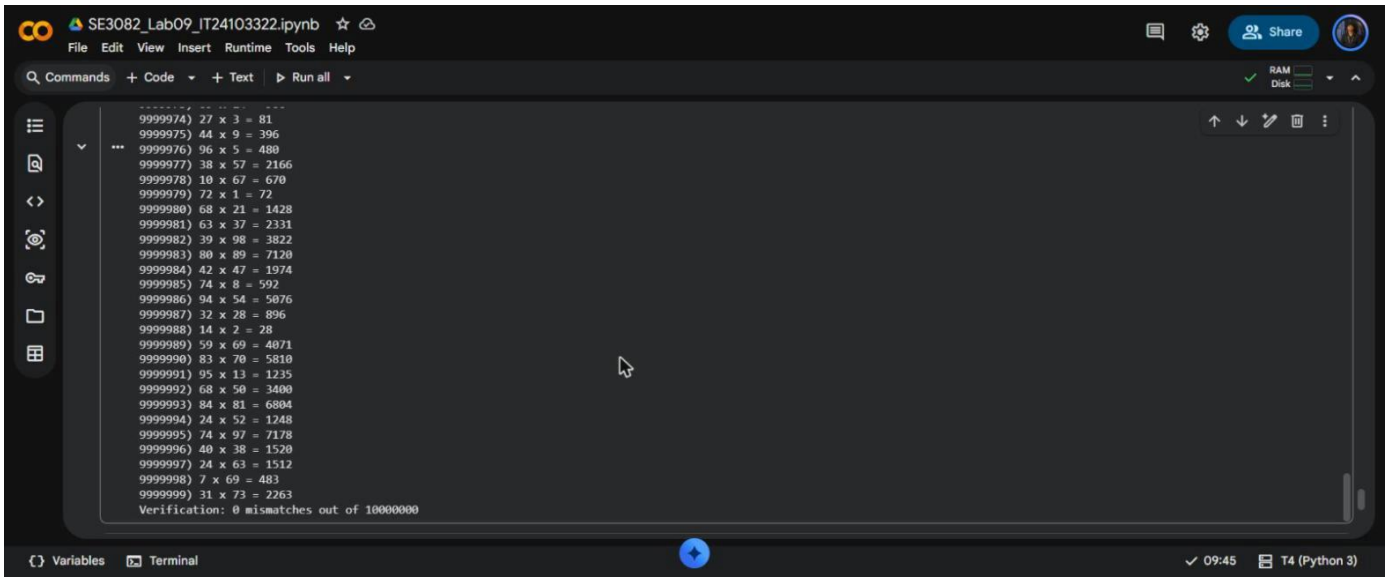
The limit is the maximum number of threads in one block, which is 1024 on the T4 (and on every current NVIDIA GPU). 2048 threads in one block is over that limit, so the launch configuration is rejected. On this CUDA 13.0 runtime the message is “invalid argument” rather than the “invalid configuration argument” wording in the sheet. N was left at 512 in ex4_threads.cu.

2. **One block of threads cannot cover a very large array.**

   Because a block can hold at most 1024 threads, one block cannot cover an array larger than that. A large array needs many blocks of many threads together, with each thread working out its global position from blockIdx.x, blockDim.x and threadIdx.x. This is what Exercise 5 does.

Exercise 5



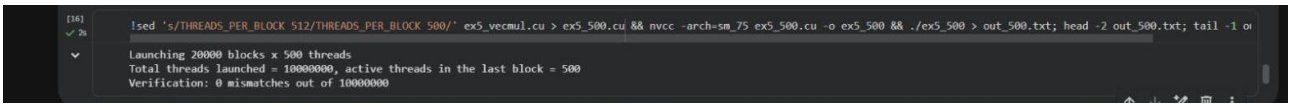


The kernel multiplies two vectors of 10,000,000 elements element by element, c[i] = a[i] * b[i], with 512 threads per block. Each thread finds its own element with the global index i = blockIdx.x * blockDim.x + threadIdx.x, and the line runs only if i < n. The number of blocks is rounded up, blocks = (N + 512 − 1) / 512 = 19532, so the threads cover every element. The last 1000 results (9999000 to 9999999) are printed, and all 10,000,000 results are checked on the CPU.

Item	Expected value	My result
Blocks launched	19532 blocks × 512 threads	Launching 19532 blocks x 512 threads
Total threads launched	10,000,384	10000384
Active threads in the last block	128	128
First of the last 1000 results	9999000) 76 x 52 = 3952	9999000) 76 x 52 = 3952
Last result	9999999) 31 x 73 = 2263	9999999) 31 x 73 = 2263
Verification	0 mismatches out of 10000000	0 mismatches out of 10000000

Every value matches the self-check. 19532 × 512 = 10,000,384 threads are launched, 384 more than the array, and only 128 threads of the last block have an element. Without the if (i < n) guard those 384 threads would read and write past the end of the arrays, which is undefined behaviour.

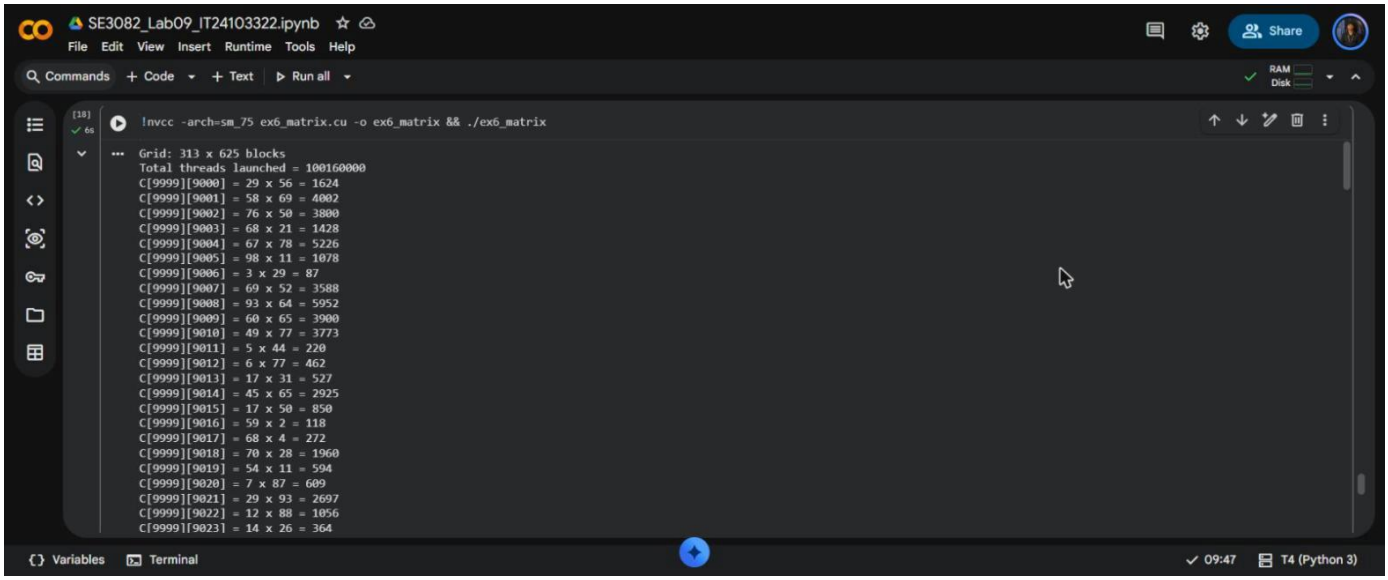
1. **Why is it a good idea to use a multiple of 32 for the threads per block? What happens to the total number of threads launched if you use 500 instead of 512?**

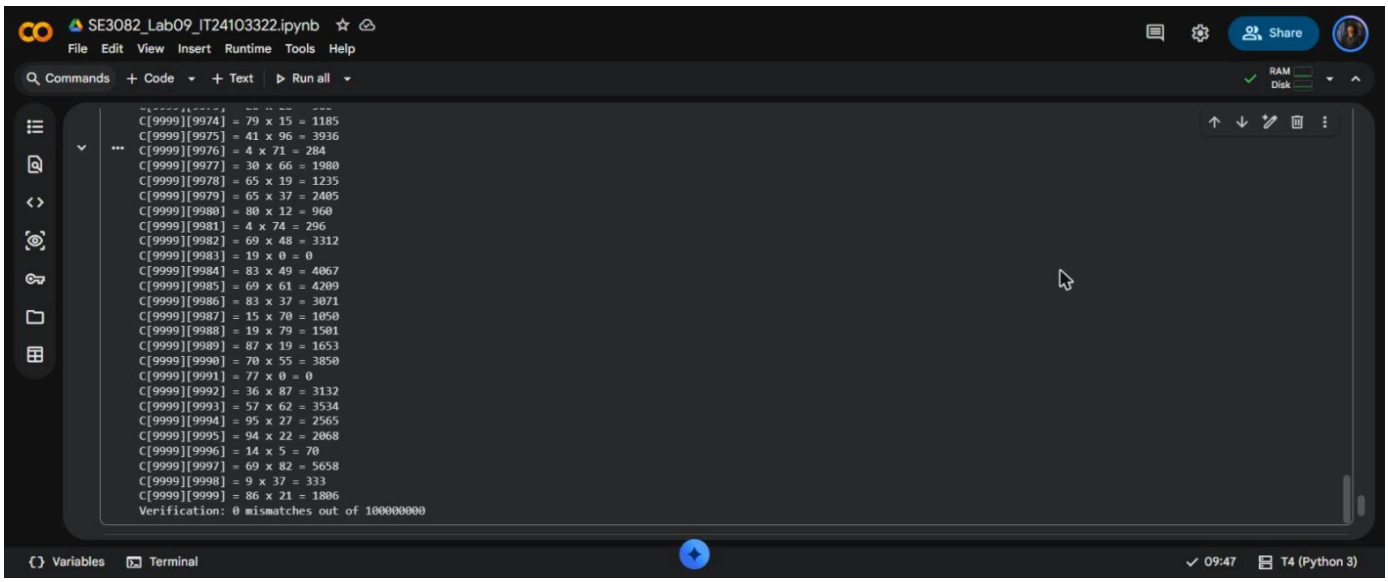


The GPU schedules threads in warps of 32. With a multiple of 32 every warp in a block is full. With 500 threads a block needs 16 warps (500 / 32 = 15.6), and the last warp has only 20 active threads and 12 idle lanes in every block.

With 500 threads per block the program launched 20000 blocks × 500 = 10,000,000 threads, exactly the size of the array, and still gave 0 mismatches. The number of threads goes down, but the hardware still runs 16 warps per block, which is 20000 × 512 = 10,240,000 lanes, compared with 10,000,384 for 512 threads per block. So 500 wastes more of the GPU even though it launches fewer threads.

Exercise 6





A, B and C are 10,000 × 10,000 matrices stored as flat row-major arrays allocated with malloc and cudaMalloc (400 MB each). Each thread computes one element, C[r][c] = A[r][c] × B[r][c]; this is an element-wise product, not matrix multiplication.

The block is dim3 threadsPerBlock(32, 16), 512 threads. The grid is rounded up in both directions: (10000 + 31) / 32 = 313 blocks across the columns and (10000 + 15) / 16 = 625 blocks down the rows. In the kernel col = blockIdx.x * blockDim.x + threadIdx.x and row = blockIdx.y * blockDim.y + threadIdx.y. The

element is computed only if row < rows and col < cols, and the flat index is idx = row * cols + col. The last 1000 elements, C[9999][9000] to C[9999][9999], are printed.

Item	Expected value	My result
Grid	313 x 625 blocks	Grid: 313 x 625 blocks
Total threads launched	100,160,000	100160000
C[9999][9000]	29 x 56 = 1624	29 x 56 = 1624
C[9999][9999]	86 x 21 = 1806	86 x 21 = 1806
Verification	0 mismatches out of 100000000	0 mismatches out of 100000000

All the values match the self-check. 313 × 625 = 195,625 blocks of 512 threads launch 100,160,000 threads, 160,000 more than the 100,000,000 elements. The extra threads are in the last block column, where 16 of the 32 threads in x are past column 9999, so both parts of the guard are needed. The run took about 6 seconds, mostly for generating the random values and the copies of 1.2 GB on the CPU side.

The notebook SE3082_Lab09_IT24103322.ipynb in my Google Drive contains the working cells for every exercise with the outputs visible.